

Figure 3.21 An SVG loaded using d3.html() that was created in Inkscape. It consists not only of the graphical `<path>` elements that make up the SVG but also much data that's often extraneous.

With the modal table we used earlier, we assumed that we pulled in all the code found in modal.html, and so we could bring it in using `d3.text()` and drop the raw HTML as text into the `.html()` function of a selection. But in the case of SVG, especially SVG that you've downloaded, you often want to ignore the verbose settings in the document, which will include its own `<svg>` canvas as well as any `<g>` elements that have been not-so-helpfully added. You probably want to deal only with the graphical elements. With our soccer ball, we want to get only the `<path>` elements. If we load the file using `d3.html()`, then the results are DOM nodes loaded into a document fragment that we can access and move around using D3 selection syntax. Using `d3.html()` is the same as using any of the other loading functions, where you designate the file to be loaded and the callback. You can see the results of this command in figure 3.21:

```
d3.html("resources/icon_1907.svg", function(data) {console.log(data);});
```

After we load the SVG into the fragment, we can loop through the fragment to get all the paths easily using the `.empty()` function of a selection. The `.empty()` function checks to see if a selection still has any elements inside it and eventually fires `true` after we've moved the paths out of the fragment into our main SVG. By including `.empty()` in a `while` statement, we can move all the path elements out of the document fragment and load them directly onto the SVG canvas.

```
d3.html("resources/icon_1907.svg", loadSVG);
function loadSVG(svgData) {
 while(!d3.select(svgData).selectAll("path").empty()) {
```

The data variable will automatically be passed to loadSVG().